

# 600086 Final Lab - Matthew Panks

---

## GPU Design

---

The kernel is split into sections to perform particle updates, collision detection, and collision handling separately, with `cudaDeviceSynchronize` used inbetween to ensure full completion of the previous section before continuing.

Global memory is used as this is accessible by all threads in all blocks without need to allocate and manage shared memory. These are allocated once on startup and de-allocated on exit.

## Update

As each thread is concerned with a single particle, locks are not required to prevent race conditions. Each thread performs acceleration, position updates, cooling, and checks if the particle collides with the walls or floor.

Particles which collide with the floor are instantly respawned to remove an additional kernel launch. Wall collisions are elastic.

```

__global__ void UpdateParticles(
    Particle* particles,
    curandState_t* states,
    float dt,
    const int* particleCount,
    uint64_t* floor_hits)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= *particleCount) return;

    Particle& p = particles[i];
    if (!p.active) return;

    // Gravity + integrate
    p.velocity.y += GRAVITY * dt;
    p.position.x += p.velocity.x * dt;
    p.position.y += p.velocity.y * dt;
    p.position.z += p.velocity.z * dt;

    p.lifetime += dt;

    // Cooling
    p.temperature -= 2.0f * dt / p.mass;
    if (p.temperature < 0.0f) p.temperature = 0.0f;

    // Floor collision / respawn: operate directly on global state
    if (p.position.y <= 0.0f) {
        curandState_t* state = &states[i];
        Kernel_RespawnParticle(p, state);
        states[i] = *state;
        atomicAdd(floor_hits, 1);
    }

    // Wall collisions
    if (p.position.x > 0.5f) { p.position.x = 0.5f; p.velocity.x *= -1.0f; }
    if (p.position.x < -0.5f) { p.position.x = -0.5f; p.velocity.x *= -1.0f; }
    if (p.position.z > 0.5f) { p.position.z = 0.5f; p.velocity.z *= -1.0f; }
    if (p.position.z < -0.5f) { p.position.z = -0.5f; p.velocity.z *= -1.0f; }
}

```

## Spatial Collision Grid

Collision detection is performed via a spatial grid, where particles in a cell are considered against the other particles in the same cell and those surrounding it.

```

__global__ void BuildGridCount(
    Particle* particles,
    int* cellCounts,
    int* nx,
    int* ny,
    int* nz,
    float* cellSize,
    int* particleCount)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= *particleCount) return;

    Particle p = particles[i];
    if (!p.active) return;

    int gx = (int)((p.position.x + 0.5f) / (*cellSize));
    int gy = (int)(p.position.y / (*cellSize));
    int gz = (int)((p.position.z + 0.5f) / (*cellSize));

    if (gx < 0 || gy < 0 || gz < 0) return;
    if (gx >= *nx || gy >= *ny || gz >= *nz) return;

    int cellIndex =
        gz * (*nx * *ny) +
        gy * *nx +
        gx;

    atomicAdd(&cellCounts[cellIndex], 1);
}

```

This simply builds the grid, which is then populated.

```

__global__ void BuildGridFill(
    Particle* particles,
    int* cellOffsets,           // READ-ONLY base offsets
    int* cellWriteOffsets,     // mutable copy (atomic cursor)
    int* cellParticleIndices,
    int* nx,
    int* ny,
    int* nz,
    float* cellSize,
    int* particleCount)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= *particleCount) return;

    Particle p = particles[i];
    if (!p.active || p.lifetime < 0.5f) return;

    // Compute cell coordinates
    int gx = (int)((p.position.x + 0.5f) / (*cellSize));
    int gy = (int)(p.position.y / (*cellSize));
    int gz = (int)((p.position.z + 0.5f) / (*cellSize));

    if (gx < 0 || gy < 0 || gz < 0) return;
    if (gx >= *nx || gy >= *ny || gz >= *nz) return;

    int cellIndex =
        gz * (*nx * *ny) +
        gy * *nx +
        gx;

    // Atomically get local offset inside this cell
    int localOffset =
        atomicAdd(&cellWriteOffsets[cellIndex], 1);

    // Compute global write index
    int writeIndex =
        cellOffsets[cellIndex] + localOffset;

    cellParticleIndices[writeIndex] = i;
}

```

Collisions are then detected by iterating the particles in each cell. Collisions are added to an array of collisions to be resolved.



```

__global__ void DetectCollisionsGPU(
    Particle* particles,
    int* cellOffsets,
    int* nx,
    int* ny,
    int* nz,
    int* cellParticleIndices,
    float* cellSize,
    Collision* collisions,
    int* collisionCount)
{
    int cellIdx =
        blockIdx.x * blockDim.x + threadIdx.x;

    int totalCells =
        (*nx) * (*ny) * (*nz);

    if (cellIdx >= totalCells)
        return;

    const int offsets[3] = { -1, 0, 1 };

    // Convert flat index
    int z = cellIdx / ((*nx) * (*ny));
    int rem = cellIdx % ((*nx) * (*ny));
    int y = rem / (*nx);
    int x = rem % (*nx);

    // Get particle range for this cell
    int start = cellOffsets[cellIdx];
    int end = (cellIdx == totalCells - 1) ? start : cellOffsets[cellIdx + 1];

    for (int i = start; i < end; i++)
    {
        int pIndex = cellParticleIndices[i];

        for (int dx : offsets)
            for (int dy : offsets)
                for (int dz : offsets)
                {
                    int nxCell = x + dx;
                    int nyCell = y + dy;
                    int nzCell = z + dz;

                    // Bounds check
                    if (nxCell < 0 || nyCell < 0 || nzCell < 0)
                        continue;

                    if (nxCell >= *nx ||
                        nyCell >= *ny ||
                        nzCell >= *nz)
                        continue;
                }
            }
        }
    }

```

```

int neighborIdx =
    nzCell * ((*nx) * (*ny)) +
    nyCell * (*nx) +
    nxCell;

int neighborStart = cellOffsets[neighborIdx];
int neighborEnd = (neighborIdx == totalCells - 1 ?
    totalCells : neighborStart + 1);

for (int j = neighborStart;
    j < neighborEnd;
    j++)
{
    if (i == j) continue;
    int qIndex =
        cellParticleIndices[j];

    // Avoid duplicate pairs
    if (pIndex >= qIndex)
        continue;

    if (!particles[pIndex].active ||
        !particles[qIndex].active)
        continue;

    // Compute distance
    float dx = particles[pIndex].position.x -
        particles[qIndex].position.x;

    float dy = particles[pIndex].position.y -
        particles[qIndex].position.y;

    float dz = particles[pIndex].position.z -
        particles[qIndex].position.z;

    float dist2 =
        dx * dx + dy * dy + dz * dz;

    const float radius = 0.01f;
    const float collisionDistance =
        radius * 2.0f;

    if (dist2 <
        collisionDistance *
        collisionDistance)
    {
        int outIndex =
            atomicAdd(&collisionCount, 1);

        if (outIndex >= MAX_PARTICLES_PER_CELL)
            return;
    }
}

```

```

        collisions[outIndex] =
        { pIndex, qIndex };
    }
}
}
}

```

Collisions are then resolved.

```

__global__ void ResolveCollisionsMerge(
    Particle* particles,
    Collision* collisions,
    int* collisionCount,
    curandState_t* states)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= *collisionCount) return;

    int a = collisions[i].a;
    int b = collisions[i].b;

    auto& pa = particles[a];
    auto& pb = particles[b];

    float m0 = pa.mass;
    float m1 = pb.mass;

    float mNew =
        m0 + m1;

    pa.velocity.x =
        (m0 * pa.velocity.x +
         m1 * pb.velocity.x) / mNew;

    pa.velocity.y =
        (m0 * pa.velocity.y +
         m1 * pb.velocity.y) / mNew;

    pa.velocity.z =
        (m0 * pa.velocity.z +
         m1 * pb.velocity.z) / mNew;

    pa.mass = mNew;

    pa.temperature =
        (m0 * pa.temperature +
         m1 * pb.temperature)
        / mNew;

    curandState* local = &states[i];
    Kernel_RespawnParticle(pb, local);
    states[i] = *local;
}

```

## Rendering

OpenGL is used to create and draw to a window, with the `cuda_gl_interop` passing particles to OpenGL to draw.

```
__global__ void BuildParticleVertices(
    Particle* particles,
    ParticleVertex* vertices,
    int* particleCount,
    int renderMode)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if (i >= *particleCount)
        return;

    Particle& p = particles[i];
    ParticleVertex v;
    v.position = p.position;

    if (renderMode == 0)
    {
        float t = p.temperature;

        v.color = make_float3(
            t,
            0.0f,
            1.0f - t);
    }
    else
    {
        if(p.mass > 4.0f)
            v.color = make_float3(
                1.0f,
                0.2f,
                0.2f);
        else if(p.mass >= 4.0f)
            v.color = make_float3(
                1.0f,
                1.0f,
                0.2f);
        else if (p.mass >= 3.0f)
            v.color = make_float3(
                0.2f,
                1.0f,
                0.2f);
        else if (p.mass >= 2.0f)
            v.color = make_float3(
                0.2f,
                0.8f,
                1.0f);
        else
            v.color = make_float3(
                0.0f,
                0.2f,
                1.0f);
    }

    vertices[i] = v;
}
```

## Synchronization

Each function is called from a launch function which handles launching, synchronization, and error handling.

Example:

```
void LaunchUpdateParticles(
    Particle* d_particles,
    curandState* d_states,
    float dt,
    const int h_particle_count,
    const int* d_particle_count,
    uint64_t* d_floorHits)
{
    const int threadsPerBlock = 256;
    const int blocks = (h_particle_count + threadsPerBlock - 1) / threadsPerBlock;

    UpdateParticles<<<blocks, threadsPerBlock>>>(d_particles, d_states, dt, d_particle_count, d_floorHits);

    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess) {
        fprintf(stderr, "UpdateParticles kernel launch error: %s\n", cudaGetErrorString(err));
        return;
    }

    cudaError_t syncErr = cudaDeviceSynchronize();
    if (syncErr != cudaSuccess) {
        fprintf(stderr, "UpdateParticles kernel execution/sync error: %s\n", cudaGetErrorString(syncErr));
        return;
    }
}
```

## CPU Design

---

Particles are initially spawned but are inactive such that they are ignored by all processes until made active by a respawn function. This prevents initial spikes in collisions when the program starts, allowing particles to increase in number and hold a steadier flow with more particles.

## Control Loop

Once relevant objects are created and data structures are populated, a thread is spawned which loops and adds threads to thread pools for physics, cooling, and spatial grid creation. This also uses channels to send the grid, receive the result of collisions, and send particles to be drawn.



```

std::thread::spawn(move || {
    let mut last = std::time::Instant::now();

    let mut emitter = Emitter {
        spawn_rate: 12000.0,
        accumulator: 0.0,
    };

    // Build initial grid for the first loop.
    build_grid(&mut grid, &particles.lock().unwrap(), &bounds);

    loop {
        let now = std::time::Instant::now();
        let dt = (now - last).as_secs_f32();
        last = now;

        let mut particles_guard = particles.lock().unwrap();
        println!("Live: {}", particles_guard.live_particles());
        //println!("Dead: {}", particles_guard.dead_particles());
        emitter.accumulator += emitter.spawn_rate * dt;
        let mut to_spawn = emitter.accumulator.floor() as usize;
        emitter.accumulator -= to_spawn as f32;

        let available = particles_guard.dead_particles() as usize;
        if to_spawn > available {
            to_spawn = available;
        }

        if to_spawn > 0 {
            particles_guard.spawn_some(to_spawn, &circle);
        }
        drop(particles_guard);

        let particles_arc = Arc::clone(&particles);
        physics_pool.install(|| {
            let mut particles_guard = particles_arc.lock().unwrap();
            physics_step(&mut *particles_guard, dt, &circle, Arc::clone(&particles_arc));
        });

        let particles_guard = particles.lock().unwrap();
        build_grid(&mut grid, &*particles_guard, &bounds);

        let snapshot = Arc::new(CollisionSnapshot::new(&*particles_guard));
        drop(particles_guard);

        let total_cells = grid.cells.len();
        let mid_cell = total_cells / 2;
        collision_job_tx.send((Arc::clone(&snapshot), 0, mid_cell));
        collision_job_tx.send((Arc::clone(&snapshot), mid_cell, total_cells));

        let particles_arc = Arc::clone(&particles);
        cooling_pool.install(move || {

```

```

        let mut particles_guard = particles_arc.lock().unwrap();
        cooling_step(&mut *particles_guard, dt);
    });

    let mut collisions = collision_result_rx.recv().unwrap();
    collisions.extend(collision_result_rx.recv().unwrap());

    merge_job_tx.send(collisions).unwrap();
    merge_done_rx.recv().unwrap();

    let particles_guard = particles.lock().unwrap();
    let mut buffer: Vec<vertex::Vertex> = Vec::new();
    particles_guard.write_positions_sampled(&mut buffer, 32);
    //println!("{}", buffer.iter().count());
    tx.send(buffer).ok();
}
});

```

## Physics Update

This chunks the Particle vector, iterating and updating each particle without needing to lock anything as each particle can only be accessed by one thread at a time.

The atomic floor counter allows the value to be updated by any thread at any time without needing to be locked. This is an arc to allow multiple ownership between threads.

```

pub fn physics_step(p: &mut Particles,
dt: f32, circle: &[f32; 2],
floor_counter: Arc<AtomicU64>,
bounds: &Bounds,
grid: &mut SpatialGrid) {
    let g: f32 = -9.81;
    // Process particles in parallel chunks
    p.particles.iter_mut().par_chunks_mut(chunk_size: 256)
        .for_each(|chunk: &mut [Particle]| {
            for particle: &mut Particle in chunk.iter_mut() {
                if !particle.alive { continue; }

                //particle.time += dt;

                // update velocity
                particle.velocity[1] += g * dt;

                // update position
                particle.position[0] += particle.velocity[0] * dt;
                particle.position[1] += particle.velocity[1] * dt;
                particle.position[2] += particle.velocity[2] * dt;

                // floor collision
                if particle.position[1] <= bounds.min_y {
                    respawn_particle(particle, circle);
                    floor_counter.fetch_add(val: 1, order: Ordering::Relaxed);
                    //particle.alive = false;
                    continue;
                }

                // boundary reflections - check position, not velocity
                if particle.position[0] <= bounds.min_x || particle.position[0] >= bounds.max_x {
                    particle.velocity[0] *= -1.0;
                }

                if particle.position[2] <= bounds.min_z || particle.position[2] >= bounds.max_z {
                    particle.velocity[2] *= -1.0;
                }
            }
        });
}

```

## Spatial Grid

Allows collision detection by inserting particles to relevant cells and checking if particles in a given cell collide with each other, or the neighbouring cells.

```

pub struct SpatialGrid {
    pub cell_size: f32,
    pub inv_cell_size: f32,
    pub dims: (i32, i32, i32),
    pub cells: Vec<Vec<usize>>,
}

impl SpatialGrid {
    pub fn new(bounds: &Bounds, cell_size: f32) -> Self {
        let nx: i32 = ((bounds.max_x - bounds.min_x) / cell_size).ceil() as i32;
        let ny: i32 = ((bounds.max_y - bounds.min_y) / cell_size).ceil() as i32;
        let nz: i32 = ((bounds.max_z - bounds.min_z) / cell_size).ceil() as i32;
        let total: usize = (nx * ny * nz) as usize;

        Self {
            cell_size,
            inv_cell_size: 1.0 / cell_size,
            dims: (nx, ny, nz),
            cells: vec![Vec::new(); total],
        }
    }

    pub fn index(&self, x: i32, y: i32, z: i32) -> usize {
        let (nx: i32, ny: i32, _) = self.dims;
        (x + y * nx + z * nx * ny) as usize
    }

    pub fn clear(&mut self) {
        for cell: &mut Vec<usize> in &mut self.cells {
            cell.clear();
        }
    }

    pub fn insert(&mut self, i: usize, x: f32, y: f32, z: f32, bounds: &Bounds) {
        let gx: i32 = ((x - bounds.min_x) * self.inv_cell_size) as i32;
        let gy: i32 = ((y - bounds.min_y) * self.inv_cell_size) as i32;
        let gz: i32 = ((z - bounds.min_z) * self.inv_cell_size) as i32;

        if gx < 0 || gy < 0 || gz < 0 { return; }
        if gx >= self.dims.0 || gy >= self.dims.1 || gz >= self.dims.2 { return; }

        let idx: usize = self.index(x: gx, y: gy, z: gz);
        self.cells[idx].push(i);
    }
}
impl SpatialGrid

```

```

pub fn build_grid(grid: &mut SpatialGrid, p: &Particles, bounds: &Bounds) {
    grid.clear();

    for (i: usize, particle: &Particle) in p.particles.iter().enumerate() {
        if particle.alive {
            grid.insert(i,
                x: particle.position[0],
                y: particle.position[1],
                z: particle.position[2],
                bounds);
        }
    }
}

```

## Collision Detection

Collision detection and handling are split into two, using channels to send detected collisions to the handler which sends the result of merges back to the controller.

```
for _ in 0..2 {
    let job_rx: Receiver<Arc<CollisionSnapshot>, ...> = collision_job_rx.clone();
    let result_tx: Sender<Vec<Collision>> = collision_result_tx.clone();

    std::thread::spawn(move || {
        while let Ok((snapshot: Arc<CollisionSnapshot>, start_cell: usize, end_cell: usize)) = job_rx.recv() {
            let collisions: Vec<Collision> = detect_collisions_snapshot(&snapshot, start_idx: start_cell, end_idx: end_cell);
            let _ = result_tx.send(collisions);
        }
    });

    let merge_particles: Arc<Mutex<Particles>> = Arc::clone(self; &particles);
    let merge_done_tx: Sender<()> = merge_done_tx.clone();
    let merge_job_rx: Receiver<Vec<Collision>> = merge_job_rx.clone();

    std::thread::spawn(move || {
        while let Ok(collisions: Vec<Collision>) = merge_job_rx.recv() {
            let mut particles: MutexGuard<'_, Particles> = merge_particles.lock().unwrap();
            let mut taken: Vec<bool> = vec![false; particles.len()];
            let mut valid: Vec<Collision> = Vec::new();

            for c: Collision in collisions {
                if !taken[c.a] && !taken[c.b] {
                    taken[c.a] = true;
                    taken[c.b] = true;
                    valid.push(c);
                }
            }

            for c: Collision in valid {
                merge(c.a, c.b, p: &mut *particles);
            }

            let _ = merge_done_tx.send(());
        }
    });
} for _ in 0..2
```

Detection is performed by iterating cells and the particles within them.

```

pub fn detect_collisions_snapshot(snapshot: &CollisionSnapshot, start_idx: usize) {
    let offsets = [-1, 0, 1];
    let (nx, ny, nz) = snapshot.grid.dims;
    let mut collisions = Vec::new();

    for cell_idx in start_idx..end_idx {
        let indices = &snapshot.grid.cells[cell_idx];
        let z = cell_idx as i32 / (nx * ny);
        let rem = cell_idx as i32 % (nx * ny);
        let y = rem / nx;
        let x = rem % nx;

        for &i in indices {
            for dx in &offsets {
                for dy in &offsets {
                    for dz in &offsets {
                        let nx_ = x + dx;
                        let ny_ = y + dy;
                        let nz_ = z + dz;

                        if nx_ < 0 || ny_ < 0 || nz_ < 0 { continue; }
                        if nx_ >= nx || ny_ >= ny || nz_ >= nz { continue; }

                        let neighbor_idx = snapshot.grid.index(nx_, ny_, nz_);

                        for &j in &snapshot.grid.cells[neighbor_idx] {
                            if i >= j { continue; }
                            if !snapshot.alive[i] || !snapshot.alive[j] { continue; }

                            let dx = snapshot.positions[i][0] - snapshot.positions[j][0];
                            let dy = snapshot.positions[i][1] - snapshot.positions[j][1];
                            let dz = snapshot.positions[i][2] - snapshot.positions[j][2];

                            let dist2 = dx*dx + dy*dy + dz*dz;
                            let radius = 0.01;
                            if dist2 < (radius * 2.0) * (radius * 2.0) {
                                if collisions.iter().count() < MAX_COLLISIONS {
                                    collisions.push(Collision { a: i, b: j });
                                }
                            }
                        }
                    }
                }
            }
        }
    }

    collisions
}

```

## Collision Handling

Detected collisions are validated before handling such that a given particle doesn't appear in two collisions and allows this lockless implementation.

```
std::thread::spawn(move || {
    while let Ok(collisions: Vec<Collision>) = merge_job_rx.recv() {
        let mut particles: MutexGuard<'_, Particles> = merge_particles.lock().unwrap();
        let mut taken: Vec<bool> = vec![false; particles.len()];
        let mut valid: Vec<Collision> = Vec::new();

        for c: Collision in collisions {
            if !taken[c.a] && !taken[c.b] {
                taken[c.a] = true;
                taken[c.b] = true;
                valid.push(c);
            }
        }

        for c: Collision in valid {
            merge(c.a, c.b, p: &mut *particles);
        }

        let _ = merge_done_tx.send(());
    }
});
```

The second particle b of a merge is made inactive or "killed" to be respawned.

```
pub fn merge(a: usize, b: usize, p: &mut Particles) {
    let m0: f32 = p.particles[a].mass;
    let m1: f32 = p.particles[b].mass;
    let m_new: f32 = m0 + m1;

    // momentum conservation
    p.particles[a].velocity[0] = (m0 * p.particles[a].velocity[0] + m1 * p.particles[b].velocity[0]) / m_new;
    p.particles[a].velocity[1] = (m0 * p.particles[a].velocity[1] + m1 * p.particles[b].velocity[1]) / m_new;
    p.particles[a].velocity[2] = (m0 * p.particles[a].velocity[2] + m1 * p.particles[b].velocity[2]) / m_new;

    p.particles[a].mass = m_new;
    p.particles[a].temp = (m0 * p.particles[a].temp + m1 * p.particles[b].temp) / (m0 + m1);

    // kill particle b
    p.particles[b].alive = false;
    p.free_indices.push(b);
}
```

## Cooling

Cooling is added to a thread pool.

```
let particles_arc: Arc<Mutex<Particles>> = Arc::clone(self: &particles);
cooling_pool.install(move || {
    let mut particles_guard: MutexGuard<'_, Particles> = particles_arc.lock().unwrap();
    cooling_step(p: &mut *particles_guard, dt)
});
```

This uses `par_iter_mut` for mutable parallel iteration

```
pub fn cooling_step(p: &mut Particles, dt: f32) {
    let cooling_rate: f32 = 2.0;
    let ambient: f32 = 0.0;

    p.particles.iter_mut().filter(|particle: &mut Particle| particle.alive).for_each(|particle: &mut Particle| {
        // Cooling depends on temp diff. between particle and ambient - more realistic
        particle.temp += ((ambient - particle.temp) * cooling_rate * dt) / particle.mass;

        if particle.temp < 0.0 {
            particle.temp = 0.0;
        }
    });
}
```

## Rendering

A sample of particle positions are written to a buffer and sent to the render loop via a channel.

```
let particles_guard: MutexGuard<'_, Particles> = particles.lock().unwrap();
let mut buffer: Vec<vertex::Vertex> = Vec::new();
particles_guard.write_positions_sampled(out: &mut buffer, step: 100);
//println!("{}", buffer.iter().count());
tx.send(buffer).ok();
```

## Performance

Temporary code has been added to each render loop to determine how long each frame takes to complete.

For fair comparison, both systems were set to use 50,000 particles and run in release mode without debugging for execution timing.

Rendering/drawing is included in the times to time the whole frame.

Each implementation will be run for around 10 seconds with the output from the last ten frames being taken to calculate an average.

These tests were performed on my personal computer with an Intel i7-12650H and Nvidia RTX 3060. This is less powerful than the fenner machines which can run this code quicker.

## Rust Execution Times

let time = Instant::now(); was added to the start of the render loop, with let duration = time.elapsed(); println!("Elapsed time: {:.2?}", duration); at the end.

```
Elapsed time: 515.20µs
Elapsed time: 2.67ms
Elapsed time: 377.30µs
Elapsed time: 540.30µs
Elapsed time: 1.04ms
Elapsed time: 523.20µs
Elapsed time: 614.70µs
Elapsed time: 404.30µs
Elapsed time: 484.50µs
Elapsed time: 2.50ms
```

Total: 9669.5us Average: 966.95us Max stable particles: 110,000

As the `Install` function of `ThreadPool` is used, threads are spawned automatically when enough work exists. Increasing partition/chunk sizes reduces the number of threads spawned while increasing work/load for each thread, causing each thread to take longer to complete. Reducing this size has the opposite effect and potentially cause extra overhead for spawning and scheduling to outweigh time reduction.

## CUDA Execution Times

`std::chrono::high_resolution_clock` was used to time this implementation, with `auto start = std::chrono::high_resolution_clock::now();` being added to the start of the render loop, and the output at the end (after rendering/drawing):

```
auto end = std::chrono::high_resolution_clock::now();
auto duration_us = std::chrono::duration_cast<std::chrono::microseconds>(end - start);
std::cout << "Execution Time: " << duration_us << " µs\n";
```

```
Execution Time: 6445 us
Execution Time: 6505 us
Execution Time: 6454 us
Execution Time: 6537 us
Execution Time: 6510 us
Execution Time: 6791 us
Execution Time: 6510 us
Execution Time: 6511 us
Execution Time: 3728 us
Execution Time: 5560 us
Execution Time: 83018 us
```

The final frame/output shall be considered an outlier for this as it is over ten times longer than all other outputs, as the simulation stutters momentarily when the close button is hovered over.

Total: 61,551us Average: 6,155.1us Max stable particles: 265,000

This uses blocks of 256 threads (8 warps per block). The GPU schedules warps not blocks, smaller blocks have lower latency and poorer latency hiding, while bigger blocks reduces the number of blocks and can reduce occupancy. 256 threads is a middle ground as smaller blocks require more registers, and larger blocks may face contention with the `atomicAdd` calls.

## Comparison

The CUDA implementation takes around six times longer to execute per frame due to having to launch and synchronize GPU kernels. However, CUDA can also handle a larger simulation with more particles due to the GPU handling all particles and collisions at the same time as each particle is handled by its own thread for physics, and cooling.

## Reflection

---

The Rust implementation went through several initial designs to find a good architecture, including an implementation which solely used channels to send and receive particles through a pipeline of functions for each step of the simulation. These faced many issues with bottlenecking and scalability, leading to the use of thread pools.

The CUDA implementation turned out to be simpler, following a process of creating an initial CPU simulation which was migrated to CUDA functions with relevant synchronises.

Each function is its own kernel with `cudaDeviceSynchronise` used as this ensures the entire grid completes before continuing. `__syncthreads()` on the device only synchronises threads within a block and causes race conditions. `cooperative_groups` were attempted but issues were faced with the grid size when scaling.

## Future Optimisations

Using a mixture of shared and global memory in CUDA could reduce memory access times for certain kernels such as collision detection due to the high number of memory reads on different values.

Certain kernels could also be merged into a single function, such as collision detection automatically handling collisions instead of passing them to another kernel.